VCEBOTT QUESTION BANK - PROJECT REPORT

Date: 2026-10-04. Status: plan awaiting build confirmation. Companion file: `PLAN.md` (decision log and technical detail).

1. Executive summary

vcebott already scrapes every VCAA Methods, Specialist and Further/General exam paper and external assessment report from 2006 to 2026, and already extracts official state-wide mark distributions for every question. That means question difficulty is already computable from authoritative data; what is missing is a way for students to browse and filter questions.

This report proposes a question bank feature: every past exam question as a cropped image, tagged with topic labels from the curriculum (`Topics.csv`), scored for difficulty from the official mark statistics, filterable by topic, difficulty, marks, exam type and year, with full-text search.

The build adds four new stages to the existing pipeline (crop, OCR, bank build, bank UI) and keeps every existing stage unchanged. The only paid component is a one-time Gemini Flash-Lite OCR pass at approximately USD 0.15 to 0.45 for both subjects; everything else runs locally and free. Topic classification is decided empirically: two candidate approaches (Flash-Lite multi-label and a local 0.8B model with thresholded probabilities) run against the same labeled sample, disagreements are manually verified, and the winner is kept while the losing path is deleted.

Total effort: approximately 3.5 to 5 development days plus one to three hours of manual topic verification by the project owner. Recurring cost: zero, because every stage is cached and the bank rebuilds from parquet without re-inference.

2. Proposed plan

2.1 What students get

- A bank page listing questions as cropped images from the real exam papers: faithful math notation, figures and tables, no re-typesetting errors.
- Filters: subject, topic (multi-select), difficulty range with a toggle between raw proportion and bank percentile, marks, exam 1 or 2, year range, separators only, and a text search box.
- Per-question detail: part-level mark distributions, share of the state scoring zero, separator gap, topic labels with confidence, and links to the original paper and assessment report.
- Difficulty badge: mean proportion of marks received (0 to 1 scale) plus a bank-normalized percentile ("harder than 71 percent of the bank").

2.2 Difficulty metric

Primary metric: `avg_proportion = average / marks_available`, the mean share of available marks the state received for that question, taken from the official mark-distribution tables. Secondary signals: percent of the state scoring zero, and separator gap (difference between the top-quartile mean and the overall mean), both already computed by the existing analysis stage.

Multiple-choice questions are binary, so their proportion equals the percent correct; the UI states this so a 0.45 on a one-mark MC question reads differently from 0.45 on a five-mark extended question. The percentile view normalizes across the whole bank for fair filtering.

Northern Hemisphere Timetable sessions are excluded: VCAA publishes no mark statistics for them (verified during parsing; they parse to zero rows).

2.3 What gets built

Stage	Status	Work
scrape, parse, analyze	exists, verified	none
crop	new	PyMuPDF: locate question headings in paper PDFs, crop to the next heading or end marker, render PNG per page segment
OCR	new	Gemini Flash-Lite REST, one image per request, temperature 0, structured JSON (latex text, plain text, optional topic labels)
classify	bake-off	Candidate B (Flash-Lite topics array) vs Candidate C (local Simple Jev probabilities with swept threshold), decided by manual verification
build-bank	new	merge marks, separators, topics, OCR text, image paths into one bank parquet per subject

Stage	Status	Work
/bank UI	new	FastAPI route plus template: filters, cards, detail view, KaTeX secondary render, search

Classification granularity is the whole question, never individual parts a, b, c; part-level statistics stay available in the detail view.

Topic vocabulary comes from `Topics.csv`: four Methods topics (Functions, relations and graphs; Algebra; Calculus; Probability and Statistics) and eight Specialist topics (Proofs and Logic; Counting; Probability and Statistics; Trigonometry; Linear Algebra; Algebra over C; Functions, relations and graphs; Calculus). Questions may carry multiple labels.

2.4 Storage

All under `data/` (gitignored, reproducible from raw sources):

- `data/raw/`: papers and reports with sha256 hashes in `manifest.json` (existing; never modified).
- `data/processed/<subject>-question-images/`: one PNG per page segment, ordered per question. Files rather than database blobs so the dashboard serves them directly as static assets.
- `data/processed/<subject>-bank.parquet`: one row per whole question with all metrics, topic lists, image paths, OCR text and nested part statistics.
- Cache parquets keyed by file sha256 and crop rectangle: interrupted or repeated stages never redo paid or slow work.

Parquet over SQLite: matches the repo's one-parquet-per-stage convention, needs no new infrastructure, and row counts are small. SQLite full-text search remains a later option if substring search proves too coarse.

3. How we get there

Phased, each phase gated by observable acceptance before the next starts.

1. **Crops.** Build the crop module with per-era heading detection rules (2006-2012 CAS-era layouts differ from modern papers). Fallback to full-page image when heading detection fails. Accept: a visual sample of 30 crops spanning 2006 to 2026 is cleanly bounded.
2. **OCR trial.** A 20-question batch through Flash-Lite including topic labels. Accept: LaTeX spot-check passes and measured cost lands within estimate. Then the full Methods run.
3. **Classification bake-off.** Remap the existing labeled fixture to the new vocabulary, run both candidates on the same set, sweep Candidate C

thresholds (0.15, 0.20, 0.25, 0.30, 0.40), and export every disagreement to a review CSV. The project owner manually verifies mismatches; the candidate that is correct more often wins and the losing path is removed at cutover. Ambiguous remapped samples are re-verified during this pass, which also grows the test set.

4. **Bank build and UI.** Merge everything into the bank parquet, then build the bank route, filters, cards and detail view for Methods. Accept: every filter combination returns consistent rows, images render, detail view matches the report data.
5. **Specialist port.** Same stages with the eight-topic vocabulary; Specialist report layouts were only smoke-tested, so layout assumptions are verified here. Accept: same checks as phases 1 to 4.
6. **Docs and cleanup.** README update, throwaway scripts removed, losing classification path deleted.

Verification approach throughout: numbers cross-checked against published VCAA averages (the existing analyze stage already reports every mismatch), row counts reconciled between marks and bank tables, and the actual UI exercised in a browser rather than unit-tested alone.

4. Cost breakdown

Paid component is OCR only. Prices at current Flash-Lite list rates; the trial batch measures actuals before the full run.

Item	Quantity	Cost (USD)
OCR trial batch	20 questions	less than 0.02
Methods OCR full pass	approximately 800 question crops	0.06 to 0.20
Specialist OCR full pass	approximately 800 question crops	0.06 to 0.20
Topics-in-OCR marginal (candidate B)	same calls, larger response	0.03 to 0.05
Contingency: Flash upgrade if OCR quality disappoints	whole bank	approximately 1.00
Compute (crops, Jev, UI, bank)	local machine	0.00
Infrastructure, hosting, database	none required	0.00
Total one-time		approximately 0.15 to 0.45 (1.45 worst case)
Recurring		0.00 (caches prevent any re-billing)

Storage: approximately 100 to 250 MB of PNG crops plus a few megabytes of parquet. The Gemini API key is environment-only, never committed.

Time is the real currency; see section 5. The cheap API is chosen precisely because the machine has no GPU: a local vision model would run hours on CPU for one-time work that costs less than a coffee.

5. Time breakdown

Development effort by phase (wall-clock, single developer):

Phase	Effort	Notes
1. Crops	0.5 to 1 day	per-era heading rules are the bulk; rendering itself is minutes for approximately 1600 crops
2. OCR trial plus full run	0.5 day effort + 0.5 to 1.5 h compute	sequential run fits rate limits; cache makes reruns free
3. Bake-off	0.5 day effort + under 0.5 h compute	Jev on CPU: assumed 1 to 5 s per question, minutes on the fixture set; plus 1 to 3 h owner manual verification
4. Bank build and UI	1 to 1.5 days	filters, cards, detail view, static file serving, KaTeX view
5. Specialist port	1 day	vocabulary plus layout verification
6. Docs and cleanup	0.25 day	
Total	3.5 to 5 dev days	plus 1 to 3 h owner verification

Calendar estimate: roughly one week including compute runs and the manual verification session. No phase blocks another user task; each phase ends usable.

Owner-involvement points, by design: crop sample review (minutes), OCR spot check (minutes), bake-off mismatch verification (1 to 3 h, this is the accuracy anchor), UI review (minutes).

6. Why this is the best option

Grounded difficulty, not proxy scoring. Difficulty comes from official VCAA state-wide mark distributions already scraped and cross-checked against

published averages by the existing pipeline. Commercial question banks score difficulty by editorial judgment or, at best, cohort accuracy on a platform population. Here every number is traceable to a published table, with the exact mismatch list VCAA itself published.

Faithful display at zero rendering risk. Cropped paper images cannot mis-render a fraction, integral bound or diagram. Re-typesetting 20 years of questions into LaTeX (the alternative) risks silent math corruption on every question; OCR text is kept only for search and as a secondary KaTeX view, never as the primary display.

Near-zero cost with zero recurring spend. The whole bank OCRs once for under half a US dollar and is cached by content hash forever. Mathpix, the strongest math OCR, would cost roughly USD 6 to 18 for OCR only and still need a separate classifier. A local vision model is free but runs hours on this GPU-less machine for a one-time job. Commercial platforms cost real subscription money and give no control over topic vocabulary.

Empirical model choice, not vendor trust. The bake-off makes classification accuracy a measured property on this curriculum's actual questions, with the owner's manual verification as ground truth, rather than trusting either a 0.8B local model or an API model blindly. Multi-label support matches how exam questions actually work (a tangent-line question is Calculus and Functions, not one or the other).

Smallest new surface on a proven pipeline. Four new stages sit on a pipeline whose data layer is already verified and cross-checked. Storage follows the existing parquet convention, so maintenance cost stays low and every artifact is reproducible from raw sources. Nothing existing breaks: current dashboard routes stay untouched.

Honest limitations, stated up front. Pre-2010 reports publish coarser statistics (some eras report only percent achieving full marks) and old PDF layouts need per-era crop tuning; NHT sessions have no statistics; exam-level percentile estimates assume question independence. Each is visible in the UI or documented, not hidden.

7. Risks and mitigations

Risk	Mitigation
Crop boundaries mis-detect across 20 years of layouts	per-era rules, page-image fallback, visual sample per decade
Flash-Lite LaTeX quality on dense set notation	20-question trial gates the full run; one-dollar Flash upgrade is the fallback

Risk	Mitigation
0.8B Jev tail-label reliability on multi-label	bake-off decides empirically; losing path deleted; low-confidence tags flagged in UI
Part and block label mismatch between reports and papers	whole-question normalization with marks-weighted aggregation; row-count reconciliation
Specialist report layouts vary	dedicated verification phase; README already marks Specialist as best-effort until proven

8. Prerequisites

- Gemini API key, supplied as an environment variable.
- One new dependency: pymupdf. No other new runtime dependencies.
- Simple Jev server only if candidate C wins the bake-off (launch command already in README).

9. Decision request

Confirm the plan as documented here and in `PLAN.md`, and the build starts at phase 1 (crop module) on confirmation.